מערכות הפעלה - 20594

סמסטר 2020ב

מרכז ההוראה: דוד שריאל
רכז תקשוב: יקיר צדוק

התחיל ב:	09:00 ,28/07/2020
מצב	הסתיים
הושלם ב-	12:04 ,28/07/2020
הזמן שלקח	3 שעות 4 דקות
ציון	**89.00** מתוך ציון מירבי של 100.00

מידע

בחינה בעקרונות מערכות הפעלה

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. **המבחן מורכב משלושה חלקים.**
2. **בחלקים א' ו ב' מופיעות שאלות פתוחות. את התשובות לכל השאלות הפתוחות יש לכתוב בתיבת טקסט .ענו תשובות מלאות בקיצור נמרץ. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות. אין לחרוג מהמקום המוקצה.**
3. **בחלק ג' (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה מבין התשובות המוצעות.**
4. **משך הבחינה - שלוש שעות.**

בהצלחה!

שאלה **1**

הושלם

9.00 נקודות
מתוך 10.00

א. (5 נק')

הסבירו מה הם ה

- **admission scheduler במערכת אצווה (batch system)**
- **memory scheduler במערכת אינטראקטיבית**
- **CPU scheduler במערכת אינטראקטיבית**

ב. (5 נק')
הסבירו רעיון של ה scheduler O(1) שהיה פעם בשימוש במערכת ההפעלה Linux.

א. admission scheduler- הרמה זו מתבצעת החלטות תזמון גסות. התזמון ברמה זאת הוא לטווח ארוך. כיוון שהעבודות צריכות להצהיר על משאבי המערכת הדרושים לה, ניתן לתכנן סדר פעולות ארוך טווח שיכול לנצל את משאבי המערכת, ולהחליט אם לקבל עוד עבודות.

memory scheduler- תזמון לטווח בינוני. התזמון ברמה זאת מעביר תהליכים ממוכן בזיכרון למוכן בדיסק או להפך ומרדום בזיכרון לרדום בדיסק ולהפך. ההחלטות ברמה זאת נועדה לשפר את נצילותשל מרכיבי החומרה.

CPU scheduler- תזמון לטווח קצר. תזמון ברמה הזאת מעביר מצב מרץ למוכן ולהפך. המתזמן מחליט את המדיניות החלפה.

ב. ה- scheduler הזה עשה את פעולותיו בסיבוכיות קבועה(!) בלי קשר לכמות התהליכים. הרעיון מאחוריו הוא כך: למתזמן יש 2 מערכים. אחד של פעילים, ושל פגים, בכל אחד מהתאים יש מצביע לרשימה של תהלכים. כל רשימה כזאת היא של אותה עדיפות וכל תא במערך הוא של עדיפות שונה. המתזמן בוחר תהליך מהרשימה עם העדיפות הגדולה ביותר. אם נגמר הזמן שהוקצע לתהליך הוא עובר לרשימת הפגים (לא בהכרח לאותה עדיפות). המתזמן ימשיך כך עד שאין יותר תהליכים במערך הזה. כשניגמרו התהליכים המערך, המתזמן משנה את המצביע של המערך להיות על מערך הפגים מה שבפועל הופך את המערך הזה להיות הפעילים ואת הישן להיות לפגים. כך תהליכים בעדיפות נמוכה לא יורעבו.

הערה:

1- המתזמן אמור להריץ את כל התהליכים

שאלה **2**

הושלם

30.00 נקודות
מתוך 30.00

א. (10 נק')

מהי בעיית היפוך עדיפויות (priority inversion problem)? כיצד היא יכולה להתעורר? כיצד ניתן להתגבר על הבעיה?

ב. (20 נק')

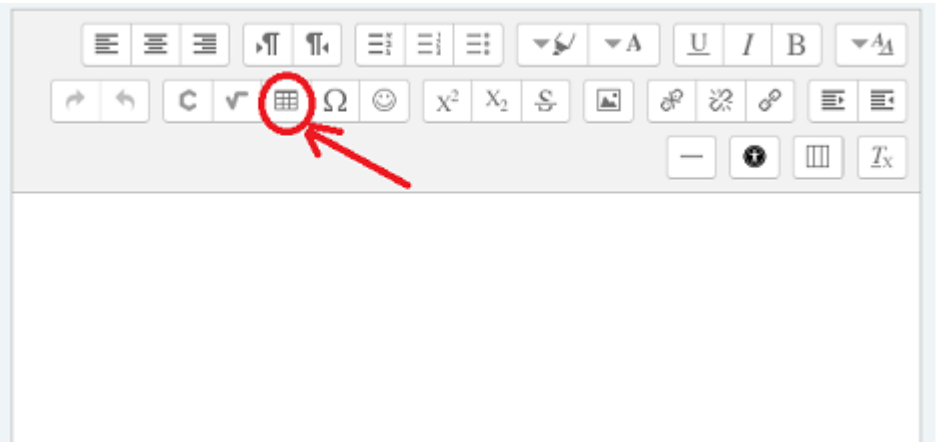
חמישה תהליכים A, B, C, D, E הגיעו למערכת בסדר אלף-ביתי בזנן t. הטבלה מטה מתארת את זמן ה CPU הנדרש לתהלכים הללו ואת עדיפותם (ערך קטן מסמל עדיפות גבוהה):

	CPU Burst	*Priority*
A	3	3
B	7	5
C	5	1
D	2	4
E	6	2

תצרו העתק של הטבלה הבאה במשבצת התשובה ומלאו בה את הפרטים החסרים: זמן המתנה לכל תהליך בהינתן מדיניות תזמון וזמן המתנה ממוצע לכל אחת ממדיניות התזמון:

Scheduling Policy	*Waiting Time*					*Average Waiting Time*
	A	*B*	*C*	*D*	*E*	
First-Come-First-Served						
Non-Preemptive Shortest-Job First						
Priority						
Round-Robin (time quantum=2)						

הבהרה: יצירת הטבלה



הבעיה היא מצב בו תהליך שעדיפותו גדולה יותר לא מצליח להתקדם ותקוע כיוון שתהליך אחר מונע זאת ממנו. הבעיה יכולה לקרות כך: תהליך א' בעל עדיפות נמוכה רק על המערכת ונכנס לקטע קריטי, במהלך הזמן שבו תהליך זה נמצא בקטע הקריטי תהליך אחר (תהליך ב') בעל עדיפות גבוהה יותר מתחיל את ריצתו. תהליך ב' כעת רוצה להיכנס לקטע הקריטי אך תהליך א' נמצא שם כעת. תהליך ב' ממשיך להחזיק את המעבד ב-busy waiting, ולא נותן הזדמנות לתהליך א' לשחרר את הקטע הקריטי. המערכת לא מתקדמת. כמובן שהבעיה יכולה לקרות כאשר יש busy waiting, ושהמתזמן נותן לתהליכים בעדיפות גבוהה לרוץ ורק כאשר הם מסיימים הוא נותן לתהליכים בעדיפות נמוכה לרוץ. פתרון הבעיה יהיה היפוך העדיפויות לזמן קצר, או שימוש במבני נתונים הנותנים לנו חסימה.

ב.

policy	a	b	c	d	e	ממוצע
fifo	3	10	15	17	23	13.6
n-p sjf	5	23	10	2	16	11.2
priority	14	23	5	16	11	13.8
RR	11	23	20	8	22	16.8

הערה:

שאלה **3**

הושלם

15.00 נקודות
מתוך 15.00

א. (6 נק')

אם יש לנו זיכרון RAM אינסופי, האם עדיין יש יתרון במנגנון זיכרון וירטואלי? הסבירו את תשובתכם.

ב. (9 נק')

נתון קטע הקוד הבא שרץ מעל מערכת ההפעלה Linux:

```
1
2  #define N SOME_NUMBER
3  #define M ANOTHER_NUMBER
4
5  int X[N];
6  for (j=0; j<30; ++j){
7     for (i = 0; i < N; i += M){
8          X[i] = X[i] + 1;
9     }
10 }
```

נניח ש TLB מתמלא באופן מעגלי ונניח כי יש בו 64 כניסות. אילו ערכים של N ושל M יגרמו ל-TLB-miss בכל איטרציה של הלולאה הפנימית?

הניחו כי TLB מתרגם כתובת דף וירטואלי לכתובת דף פיזי וגודל הדף הוא 4KB.

א. יש עדיין יתרון בזיכרון וירטואלי גם כאשר הRAM הוא אינסופי. שימוש בזיכרון וירטואלי מאפשר למשתמש להתייחס לתוכנית כשמורה באופן רציף בזיכרון אך המערכת מנהלת מאחורה את הפניות לזיכרון. המשתמש עוד עם כתובות רציפות ווירטואליות שמומרות ברקע לכתובות פיזיות.

ב. בTLB יש 64 דפים, אנחנו צריכים שבכל סיום ריצה של הלולאה החיצונית נפנה את הדף הראשון שהלולאה הזאת ניגשה אליו. לכן אני רוצים לרוב על לפחות 65 דפים(N>64), כן בסוף הריצה דף 1 לא יהיה בטבלת TLB. כעת נדאג שכל ריצה של הלולאה הפנימית תיבא לTLB דף חדש, כיוון שמערך נשמר באופן רציף בזיכרון הויראטולי אנחנו יודעים שקפיצה בגודל של דף קדימה תקדם אותנו לדף הבא. נחשב מספר האינטים בדף:

M=2^12 [byte]/(sizeof(int)[byte])

קיבלנו גודל חסר יחידות שמתאר את מספר הפעמים שאינט נכנס בדף, כל פעם נקפוץ בו אז נעבור לדף הבא ולכן נצטרך להביא לTLB דף חדש.

הערה:

שאלה **4**

הושלם

0.00 נקודות
מתוך 5.00

הוכיחו כי בפתרון של Peterson זמן ההמתנה של תהליך המעוניין להיכנס לקטע קריטי אינו עולה על הזמן אשר לוקח לתהליך השני על מנת להיכנס ולעזוב את הקטע הקריטי.

הערה: מדובר על פתרון Peterson לשני תהליכים.

התהליך מגיע לשלב שהוא רוצה להכנס לקטע הקריטי הוא משנה את הרצון שלו להיות TRUE, ומשנה את המשתנה TURN להיות המספר הסידורי שלו. כעת יש שני מצבים או שיש תהליך בקטע הקריטי או שלא. אם יש אז ערך הרצון של אותו תהליך יהיה TRUE, והתהליך הראשון ישאר בלולאה עד אשר התהליך השני יצא מהקטע הקריטי ושינה את הערך הרצון שלו להיות FALSE, מה שיגרור יציאה מהלולאה האין סופית וכניסה לקטע הקריטי של התהליך הראשון. אם אין תהליך בקטע הקריטי הוא יכנס אליו פשוט. במידה והתהליך שתפס את הקטע הקריטי בגלל תנאי הלולאה כי TURN יהיה בערך של התהליך שהגיע עכשיו וערך הרצון של התהליך השני TRUE, וכך התהליך שחיכה יכנס ברגע שקיבל זמן מעבד. תנאי הלולאה גורם לכך שהתהליך שהגיע קודם יכנס ברגע שיתפנה הקטע הקריטי והוא זה שקיבל את המעבד.

הערה:
אך ייתכן שתהליכון יצא מקטע קריטי ומיד באותו הקוונטום ינסה להיכנס שוב לקטע קריטי, ישנה את interested[i]=true, וייכנס שוב כאשר התהליכון השני אפילו לא קיבל קוונטום בכדי להתקדם. בפרט, ייתכן כי המתזמן יחליט לתזמן תהליכון אחד מספר פעמים בעוד השני לא יקבל מהמתזמן זמן CPU כלל. במקרה כזה מה מונע מהתהליכון להיכנס שוב ושוב?

שאלה **5**

הושלם

4.00 נקודות
מתוך 5.00

מה מתבצע בשורות 385-396 של פונקציית bmap בקובץ fs.c של מערכת ההפעלה xv6?
בפרט האם בשורה 393 תוכן הבלוק bp נכתב לדיסק באופן מיידי?

```
365  // The content (data) associated with each inode is stored
366  // in blocks on the disk. The first NDIRECT block numbers
367  // are listed in ip->addrs[].  The next NINDIRECT blocks are
368  // listed in block ip->addrs[NDIRECT].
369
370  // Return the disk block address of the nth block in inode ip.
371  // If there is no such block, bmap allocates one.
372  static uint
373  bmap(struct inode *ip, uint bn)
374  {
375    uint addr, *a;
376    struct buf *bp;
377
378    if(bn < NDIRECT){
379      if((addr = ip->addrs[bn]) == 0)
380        ip->addrs[bn] = addr = balloc(ip->dev);
381      return addr;
382    }
383    bn -= NDIRECT;
384
385    if(bn < NINDIRECT){
386      // Load indirect block, allocating if necessary.
387      if((addr = ip->addrs[NDIRECT]) == 0)
388        ip->addrs[NDIRECT] = addr = balloc(ip->dev);
389      bp = bread(ip->dev, addr);
390      a = (uint*)bp->data;
391      if((addr = a[bn]) == 0){
392        a[bn] = addr = balloc(ip->dev);
393        log_write(bp);
394      }
395      brelse(bp);
396      return addr;
397    }
398
399    panic("bmap: out of range");
```

בשורות האלו אנחנו בודקים מה הכתובת פיזית של הכתובת הוירטואלית בתור הSINGLE INDIRECT. תחילה אנו בודקים אם הוא נמצא בכלל בו. אם כן נבדוק אם יש בדיסק כתובת לבלוק הזה, אם אין נבצע אלוקציה בדיסק. לאחר מכן קוראים את הבלוק לבאפר עושים קאסטין כדי שנוכל לקפוץ לפי גודל של כתובת. אם אין בלוק בכתובת הרצויה בדיסק תבצע אלוקציה בדיסק ותעדכן את הלוג

שיבצע עדכון. לאחר מכן נשחרר את הבלוק ונחזיר את הכתובת. הכתיבה לא מתבצעת באופן מיידי אלא על ידי הLOG.

הערה:
הכתובת הוירטואלית ??
-1

שאלה **6**

הושלם

5.00 נקודות
מתוך 5.00

הסבירו מה מתבצע בתוכנית הבאה:

```
12  main(void)
13  {
14      int n, fd[2];
15      pid_t   pid;
16      char    line[2048];
17
18      if (pipe(fd) < 0)
19          fprintf(stderr,"pipe error");
20
21      if ( (pid = fork()) < 0)
22          fprintf(stderr,"fork error");
23
24      else if (pid > 0) {             /* parent */
25          close(fd[0]);
26
27          /* redirection of stdout so that stdout
28                     of the process would be actualy
29             written to the pipe
30          */
31                  close(1); /* stdout */
32          dup(fd[1]);
33          close(fd[1]);
34          write(1, "hello world\n", 12);
35
36      } else {                /* child */
37          close(fd[1]);
38
39          /* redirection of stdin so that stdin would
40             be actualy read from pipe
41          */
42          close(0); /*stdin*/
43          dup(fd[0]);
44          close(fd[0]);
45          n = read(0, line, 2048);
46          write(1, line, n);
47      }
48
49      exit(0);
50  }
```

כפי שהוצג בחומר רגע לממ"ן 12, התוכנית מייצרת PIPE ואז מייצרת בן. לאחר מכן בתהליך האב התוכנית סוגרת את ה STDOUT (פייל דיסקריפטור של זה) של האב ובמקומו שמה את הPIPE (פייל דיסקריפטור), ואז מדפיסה את המחרוזת "HELLO WORLD" לתוך הפייפ. בתהליך הבן הפונקציה סוגרת את STDIN (פייל

דיסקריפטור של זה) ובמקומות שמה את הPIPE (פייל דיסקריפטור), לאחר מכן היא קוראת ממנו את המידע ומדפיסה לSTDOUT, במקרה שלנו "HELLO WORLD". התוכנית מתייחסת גם למקרים שיצירת הבן או הPIPE לא צלחה.

הערה:

שאלה **7**

הושלם

3.00 נקודות
מתוך 5.00

להכנת ממ"ן 11 קראנו מאמר בו תואר כיצד הוספת תהליכונים כספריה בשפת C גורמת למספר בעיות אשר דורשות התיחסות. לדוגמא, אם

```
struct { int a:17; int b:15 } x;
```

אז x.a = 42 יכולה להיות מתורגמת ע"י מהדר (compiler) ל

```
{
  tmp = x; // Read both fields into
           // 32-bit variable.
  tmp &= ~0x1ffff; // Mask off old a.
  tmp |= 42;
  x = tmp;  // Overwrite all of x.
}
```

הסבירו כיצד הדבר יכול להשפיע על תוצאות הרצת תהליכונים.

הפעולה שמיוצגת היא עידכון של הערך של A להיות 42. מה שעושים זה מעתיקים את המספר הישן למשתנה זמני, בו מעדכנים את הערך של הביטים של A ואז מעתיקים את הערך של TMP ל-X. הבעיה יכולה לנבוע אם תהליכון מעדכן את B במהלך הפעולה שמתקיימת פה. הפעולה שמוצגת מתחילה ומעתיקה את X לTMP. לאחר מכן יש מעבר לתהליכון השני שמעדכן את B בX. כעת חוזרים לתהליך הזה וממשיכים כרגיל, כיוון שכבר העתקנו את X לTMP אז הוא לא התעדכן לאחר העדכון של B. ולכן כאשר נעתיק בסוף את TMP לX נגרום לכך שהשינוי לB לא התבצע.

הערה:
עבור הקוד לדוגמה חסר הסבר קונקרטי כיצד זה יכול להתרחש. דריסת מידע ע"י שני תהליכונים שמבצעים x.a=42 אינה רלוונטית משום שדריסה זו קוראת גם כאשר מדובר בתהליכונים הנתמכים בשפה וזו בעיה ידועה של גישה במקביל למשתנה משותף ואינה קשורה לשאלה הספציפית כאן. השאלה כאן היא לא גישה במקביל למשתנה משותף אלא גישה למשתנים שונים שאין בינהם קשר.
התהליכון השני ניגש למשתנה x.b ולא x.a. בנוסף, הקומפיילר מתרגם את הקוד הנ"ל באותה מידה גם כאשר מדובר בתהליכונים מהשפה, זו הדרך לתרגם את הקוד! , נזכור שהמתכנת יודע שלמשתנה הזה אסור לגשת במקביל ולכן הוא יגן בקטע קריטי על שורות קוד אלו מגישה במקביל. מה שנדרש בשאלה זה להראות כיצד עבור גישה למשתנה אחר בכלל, בקוד אחר, יתכן רמיסה. במקרה כזה המתכנת אינו יכול לדעת שצריך מניעה הדדית בין קטעי קוד שונים ומשתנים שונים כי זו שגיאה שלא קיימת בקוד אלא תלויית קומפיילר

שאלה **8**

הושלם

3.00 נקודות
מתוך 5.00

הסבירו מהו binary translation. האם לשיטה זו קיים עתיד? נמקו את תשובתכם.

היא שיטה שמתרגמת קוד בינארי המצריך קריאות מערכת (או קריאות אחרות לא בטוחות) בין המערכת ההפעלה האורחת לפעולות מתאימות במערכת ההפעלה המארחת. ישנם שני סוגים של תרגום סטטי שקורה מראש ודינאמי שקורה בזמן ריצה. לשיטה יש עתיד כיוון ומשתמשים בה כעת במערכות שמבצאות וירטואליזציה.

הערה:
המצריך קריאות מערכת (או קריאות אחרות לא בטוחות) ??!! XX

שאלה **9**

תקין

5.00 נקודות
מתוך 5.00

מהי הסיבה העיקרית לשימוש ב DMA ?

יש לבחור תשובה אחת:

- DMA מאפשרת למעבד לרוץ בקצב מהיר יותר.
- DMA מאפשר למעבד להיות פנוי יותר. ✓
- DMA מאפשר לדיסק להיות פנוי יותר.
- DMA מספקת מעין חוצץ (buffer) לדיסקים איטיים.

תשובתך נכונה.

שאלה **10**

תק'ין

5.00 נקודות
מתוך 5.00

במערכת ניהול זיכרון מדפדף נהוגה מדיניות של prepaging – הבאת מספר כלשהו של דפים השייכים לתהליך בכל פעם שהתהליך עובר מהדיסק לזיכרון. בחרו טענה נכונה:

יש לבחור תשובה אחת:

- **הבאת קבוצת דפים שנבחרה בקפידה היא פעולה שיכולה להוריד את כמות פסיקות הדפים (page faults) במערכת** ✓
- **אין טעם להביא דפים מראש שכן הדבר כרוך בפעולת פינוי בשלב מאוחר יותר**
- **מדיניות זו עומדת בסתירה לעקרון קבוצת העבודה**
- **מדיניות זו ניתנת למימוש רק בשילוב עם האלגוריתם האופטימאלי להחלפת הדפים**

תשובתך נכונה.

שאלה **11**

תקין

5.00 נקודות
מתוך 5.00

כדי לבחור את הדף המתאים לפינוי מהזיכרון הראשי הוצע האלגוריתם הבא:

לכל מסגרת (frame) בזיכרון מוצמד מונה. כאשר דף חדש מגיע למסגרת, ערכו של מונה זה נקבע להיות 0 ובכל פניה לדף המונה מועלה ביחידה אחת. כאשר יש צורך לפנות דף מהזיכרון, בוחרים תמיד את הדף שהמונה מוצמד למסגרת שלו מכיל את הערך הנמוך ביותר. האם אלגוריתם זה הוא טוב לניהול דפדוף בזיכרון?

יש לבחור תשובה אחת:

- **כן. המונה משקף את מידת נחיצותו של הדף. ככל שערכו נמוך, כך מספר הפניות לדף היה נמוך יותר ולכן הוא מתאים לפינוי.**
- **כן. מונה גבוהה מצביע על דף הנמצא בשימוש מתמיד.**
- **לא. אלגוריתם זה אינו מאפשר כלל כניסה של דפים חדשים כי המונה שלהם תמיד 0 ולכן יפונו מיד.**
- **לא. מצב המונה עלול לשקף מציאות היסטורית שאינה קיימת עוד.** ✓

תשובתך נכונה.

שאלה **12**

תקין

5.00 נקודות
מתוך 5.00

בחרו מידע אשר איננו מצוי בprocess control block של תהליך

יש לבחור תשובה אחת:

- מצב של תהליך (ריצה, המתנה וכו')
- מצביעה לטבלת קבצים פתוחים אשר נפתחו ע"י התהליך
- מצביע למפת הזיכרון של התהליך
- מצביע למפת הבלוקים הפנויים בדיסק ✓

תשובתך נכונה.